

2.1.a

Может ли внутри методов `Lock.lock()` или `Lock.unlock()` произойти исключение?

Ответ

Да, возможно.

Несмотря на то, что интерфейс `Lock` не объявляет исключений, в нем могут возникнуть непроверяемые исключения, поскольку в спецификации JLS сказано что на этапе компиляции проверяются только `checked`-исключения.

В JLS **параграф 11.2** указано, что только проверяемые исключения должны быть объявлены в сигнатуре метода или обработаны. Непроверяемые исключения (`RuntimeException`) и ошибки JVM (`Error`) не проверяются компилятором.

Следовательно, внутри методов `lock()` или `unlock()` могут возникнуть:

- `RuntimeException`
- `OutOfMemoryError`
- `StackOverflowError`
- и другие подклассы `VirtualMachineError`

Кроме того, в JLS **параграф 11.1.3** сказано, что асинхронные исключения могут возникать в произвольные моменты выполнения программы.

Таким образом, Java не гарантирует, что выполнение `lock()` или `unlock()` не будет прервано исключением.

2.1.b

Возможно ли реализовать устойчивые к исключениям примитивы синхронизации в Java?

Ответ

Нет, это невозможно. Даже если реализация `lock()` и `unlock()` не выбросит проверяемое исключение, невозможно предотвратить возникновение `Error`, например:

- `OutOfMemoryError`
- `StackOverflowError`
- `InternalError`
- и любые другие ошибки виртуальной машины

в JVM существуют механизмы предоставляющие частичную устойчивость к таким сбоям ВМ. Например, **ЖЕР 270** было введено выделение области стека для критически секций, что уменьшало вероятность `StackOverflowError` до освобождения ресурса. Однако даже это полностью не защищало примитивы синхронизаций от исключений, поскольку существуют такие ошибки как, `OutOfMemoryError`, `InternalError` или аварийное завершение JVM.

Вывод

вJava нет механизма гарантирующего, что критическая секция завершится корректно при любых исключениях, поскольку ошибки виртуальной машины могут возникнуть в любой точке исполнения программы, и эти исключения могут быть не обработаны компилятором.

Можно повысить устойчивость через `try-finally`, но полностью “bullet-proof” примитивы синхронизации в java невозможны.